Functions in Python

Technical Notes

Rodrigo Kang

Table of contents

1 Functions

A **function** is a reusable unit of code designed to perform a specific task. It can receive input values, perform a sequence of operations, and optionally return one or more results.

Functions provide one of the fundamental mechanisms for **abstraction** in programming. Instead of repeatedly describing *how* an operation is performed, we can define the operation once, give it a meaningful name, and subsequently use that name whenever the operation is required.

For example, suppose that we repeatedly need to convert temperatures from Celsius to Fahrenheit. The conversion is

$$F = \frac{9}{5}C + 32. \tag{1}$$

We could write the corresponding expression every time a conversion is required:

```
celsius = 20
fahrenheit = (9 / 5) * celsius + 32
fahrenheit
```

```
68.0
```

However, if this operation is required repeatedly, it is preferable to represent it as a function:

```
def celsius_to_fahrenheit(celsius):
    return (9 / 5) * celsius + 32
```

The conversion can then be performed simply by calling the function:

```
celsius_to_fahrenheit(20)
```

```
68.0
```

The expression that implements the conversion has not disappeared. Rather, its implementation has been encapsulated behind the name celsius_to_fahrenheit.

This distinction becomes increasingly important as programs grow in complexity. A well-designed function allows us to reason about **what an operation does** without having to consider its implementation every time it is used.

Functions therefore contribute to several important properties of a program:

- **Abstraction:** implementation details can be hidden behind a meaningful interface.
- **Reusability:** the same operation can be performed without duplicating code.
- **Modularity:** complex problems can be decomposed into smaller and more manageable units.

- **Readability:** meaningful function names can express the intent of a computation.
- **Testability:** individual pieces of behaviour can be tested independently.

These properties are particularly important in data science and machine learning, where a workflow often consists of a sequence of transformations such as loading data, preprocessing features, fitting models, computing metrics, and generating predictions.

1.1 Defining Functions

User-defined functions in Python are normally created using the def statement. The general structure is

```
def function_name(parameters):
    # function body
    return result
```

A function definition contains several elements:

- def indicates that a function is being defined.
- function_name identifies the function.
- parameters specify the inputs accepted by the function.
- The indented block constitutes the **function body**.
- return, when present, terminates the function and sends a value back to the caller.

For example:

```
def square(x):
    result = x ** 2
    return result
```

Here, square is the function name, x is a parameter, and result is a local variable created during the execution of the function.

The function definition itself does not calculate a square. It creates a function object and binds it to the name square. The body is executed only when the function is called:

```
square(5)
```

```
25
```

The value 5 is passed to the function, the expression x ** 2 is evaluated, and the resulting value is returned to the caller.

For a simple function such as this one, the intermediate variable is unnecessary:

```
def square(x):
    return x ** 2
```

The two implementations produce the same result. The second is generally preferable when the intermediate variable does not improve readability or serve another purpose.

1.2 Parameters and Arguments

The terms **parameter** and **argument** are closely related but refer to different things.

A **parameter** is a name appearing in the function definition:

```
def power(base, exponent):
    return base ** exponent
```

Here, base and exponent are parameters.

An **argument** is an actual value supplied when the function is called:

```
power(2, 3)
```

```
8
```

Here, 2 and 3 are arguments. During the call, they are associated with the parameters base and exponent, respectively.

This distinction is useful when reasoning about function interfaces:

> Parameters belong to the function definition; arguments belong to a particular function call.

1.2.1 Positional Arguments

Arguments can be supplied according to their position in the function definition.

Consider

```
def divide(numerator, denominator):
    return numerator / denominator
```

The call

```
divide(10, 2)
```

```
5.0
```

associates 10 with numerator and 2 with denominator.

The order matters:

```
divide(2, 10)
```

```
0.2
```

Although both calls are valid, they represent different computations.

1.2.2 Keyword Arguments

Arguments can also be associated explicitly with parameter names:

```
divide(numerator=10, denominator=2)
```

```
5.0
```

When keyword arguments are used, their order does not determine the association:

```
divide(denominator=2, numerator=10)
```

```
5.0
```

Both calls therefore produce the same result.

Keyword arguments can make function calls easier to understand, particularly when a function accepts several parameters or when the meaning of a value would otherwise be ambiguous.

For example,

```
train_model(data, 100, 0.01)
```

provides relatively little information about what 100 and 0.01 represent. A call such as

```
train_model(data, epochs=100, learning_rate=0.01)
```

communicates considerably more intent.

This is one reason why keyword arguments are common in machine learning libraries, where estimators and training procedures frequently expose many configurable parameters.

1.2.3 Default Parameter Values

A parameter can be assigned a default value in the function definition:

```
def power(base, exponent=2):
    return base ** exponent
```

If no argument is provided for exponent, the default value is used:

```
power(5)
```

```
25
```

An explicit argument overrides the default:

```
power(5, 3)
```

```
125
```

Default parameters are useful when a function has a sensible standard behaviour but should allow that behaviour to be configured.

Parameters without default values must appear before parameters with default values:

```
def function(required_parameter, optional_parameter=10):
    ...
```

This design separates the inputs that the caller must provide from those for which the function already defines a reasonable default.

A particularly important detail concerns **mutable default arguments**. Consider the following function:

```
def append_value(value, values=[]):
    values.append(value)
    return values
```

At first sight, we might expect each call without values to start with a new empty list. However:

```
append_value(1)
```

```
[1]
```

```
append_value(2)
```

```
[1, 2]
```

The second result contains the value added during the previous call.

This happens because default argument expressions are evaluated when the function is **defined**, not every time it is called. The same list is therefore reused across calls.

A common pattern is to use None as a sentinel and create the mutable object inside the function:

```
def append_value(value, values=None):
    if values is None:
        values = []

    values.append(value)
    return values
```

Now each call that omits values creates a new list:

```
append_value(1)
```

```
[1]
```

```
append_value(2)
```

```
[2]
```

This behaviour is an important Python detail because mutable default arguments can introduce state that persists between function calls even when that persistence was not intended.

1.2.4 Arbitrary Positional Arguments: *args

Sometimes the number of positional arguments that a function should accept is not known in advance. Python allows additional positional arguments to be collected using the * syntax:

```
def add_all(*numbers):
    return sum(numbers)
```

The function can then receive any number of positional arguments:

```
add_all(1, 2)
```

```
3
```

```
add_all(1, 2, 3, 4, 5)
```

```
15
```

Inside the function, numbers is a tuple containing the supplied arguments:

```
def inspect_args(*args):
    print(args)
    print(type(args))

inspect_args(10, 20, 30)
```

```
(10, 20, 30)
<class 'tuple'>
```

The name args is conventional rather than special. The * is what causes the positional arguments to be collected.

For example, the following definition is equally valid:

```python
def add_all(*values):
    return sum(values)
```

Nevertheless, *args is the standard convention when no more descriptive name is appropriate.

1.2.5 Arbitrary Keyword Arguments: **kwargs

Similarly, additional keyword arguments can be collected using **:

```python
def describe_model(**settings):
    return settings
```

For example:

```python
describe_model(
    model="logistic_regression",
    penalty="l2",
    max_iter=1000
)
```

```
{'model': 'logistic_regression', 'penalty': 'l2', 'max_iter': 1000}
```

Inside the function, settings is a dictionary whose keys are the argument names and whose values are the corresponding argument values.

```python
def inspect_kwargs(**kwargs):
    print(kwargs)
    print(type(kwargs))

inspect_kwargs(alpha=0.1, max_iter=1000)
```

```
{'alpha': 0.1, 'max_iter': 1000}
<class 'dict'>
```

Again, kwargs is only a naming convention. The ** syntax is what collects additional keyword arguments.

Functions may combine ordinary parameters, *args, and **kwargs. For example:

```
def inspect_call(x, *args, **kwargs):
    print("x:", x)
    print("args:", args)
    print("kwargs:", kwargs)

inspect_call(10, 20, 30, method="mean", normalize=True)
```

```
x: 10
args: (20, 30)
kwargs: {'method': 'mean', 'normalize': True}
```

Understanding *args and **kwargs is particularly useful when working with libraries and frameworks that expose flexible APIs or when writing functions that delegate arguments to other functions.

1.3 Return Values

A function communicates its result to the caller through the return statement.

For example:

```
def mean(a, b):
    return (a + b) / 2

result = mean(10, 20)
result
```

```
15.0
```

When Python reaches return, execution of the function terminates immediately and the specified object is returned.

Consider:

```
def absolute_value(x):
    if x < 0:
        return -x

    return x
```

If $x < 0$ is true, the first return terminates the function. The final return is therefore reached only when x is non-negative.

1.3.1 Returning Multiple Values

A Python function can appear to return several values:

```
def quotient_and_remainder(a, b):
    quotient = a // b
    remainder = a % b

    return quotient, remainder
```

For example:

```
quotient_and_remainder(41, 7)
```

```
(5, 6)
```

Strictly speaking, the function returns **one object**: a tuple containing the two values.

```
result = quotient_and_remainder(41, 7)

type(result)
```

```
tuple
```

Tuple unpacking can then assign the individual components to separate variables:

```
quotient, remainder = quotient_and_remainder(41, 7)

print(quotient)
print(remainder)
```

```
5
6
```

This pattern is common in scientific computing and data analysis, where a computation may naturally produce several related outputs.

1.3.2 Functions Without an Explicit return

A function does not need to contain an explicit return statement:

```
def greet(name):
    print(f"Hello, {name}!")
```

Calling the function produces the intended side effect:

```
greet("Alice")
```

```
Hello, Alice!
```

However, every Python function returns a value. If execution reaches the end of the function without encountering an explicit return, Python returns None:

```
result = greet("Alice")

print(result)
```

```
Hello, Alice!
None
```

Therefore, it is more precise to say that a function has **no explicit return value** than to say that it "returns nothing."

This distinction will become important when considering the difference between functions that **compute and return values** and functions whose primary purpose is to produce **side effects**, such as printing output or modifying an external object.

1.4 Scope and Name Resolution

Variables created inside and outside a function do not necessarily belong to the same **scope**.

A scope determines the region of a program in which a particular name can be accessed directly. Understanding scope is important because functions introduce their own local namespace and therefore provide a degree of isolation between different parts of a program.

Consider:

```
x = 10

def show_value():
    y = 20
    print(x)
    print(y)

show_value()
```

```
10
20
```

The variable x is defined outside the function and is therefore available in the global scope. The variable y, by contrast, is created when show_value() executes and belongs to the function's local scope.

Attempting to access y outside the function produces an error:

```
print(y)
```

Python cannot find the name y in the surrounding scope because the local namespace associated with the function call no longer provides that name to the caller.

1.4.1 Local and Global Names

A name assigned inside a function is normally treated as local to that function.

For example:

```
value = 10

def change_value():
    value = 20
    print("Inside:", value)

change_value()
print("Outside:", value)
```

```
Inside: 20
Outside: 10
```

The assignment inside change_value() does not modify the global value. Instead, it creates a new local name that temporarily **shadows** the global one.

The two names happen to have the same spelling, but they belong to different scopes.

This distinction allows functions to use internal variables without necessarily affecting variables with the same names elsewhere in the program.

1.4.2 The LEGB Rule

When Python evaluates a name, it searches for that name through a hierarchy of scopes commonly summarized by the acronym **LEGB**:

1. **Local**
2. **Enclosing**
3. **Global**
4. **Built-in**

Python stops at the first matching name it finds.

The **local** scope corresponds to the currently executing function:

```
def example():
    x = "local"
    print(x)

example()
```

```
local
```

An **enclosing** scope arises with nested functions:

```
def outer():
    x = "enclosing"

    def inner():
        print(x)

    inner()

outer()
```

```
enclosing
```

Here, inner() does not define x locally, so Python searches the enclosing scope created by outer().

The **global** scope corresponds to names defined at the module level:

```
x = "global"

def example():
    print(x)

example()
```

```
global
```

Finally, the **built-in** scope contains names provided by Python itself, such as len, sum, min, max, and print:

```
len([10, 20, 30])
```

```
3
```

This search order explains why redefining built-in names is usually undesirable:

```
sum = 10
```

After this assignment, the name sum no longer refers to Python's built-in sum() function in that scope.

For this reason, names such as list, dict, str, sum, max, and min should generally not be used for ordinary variables.

1.4.3 The global Statement

If a function needs to rebind a name in the global scope, Python provides the global statement:

```
counter = 0

def increment():
    global counter
    counter += 1

increment()
increment()

counter
```

```
2
```

Without global, an assignment to counter inside the function would make counter a local name.

Although global is occasionally useful, routinely modifying global state from inside functions can make programs harder to understand and test. In many cases, explicitly passing values into a function and returning the result provides a clearer design:

```
def increment(counter):
    return counter + 1

counter = 0
counter = increment(counter)
counter = increment(counter)

counter
```

```
2
```

The second implementation makes the flow of data explicit: the function receives a value and returns a new one instead of modifying external state.

1.4.4 The nonlocal Statement

Nested functions can also rebind names defined in an enclosing function using nonlocal.

For example:

```
def make_counter():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count

    return increment
```

The returned function retains access to the enclosing count variable:

```
counter = make_counter()

counter()
counter()
counter()
```

```
3
```

The nonlocal statement tells Python that count should not be treated as a new local variable inside increment(). Instead, assignments should affect the name in the nearest enclosing function scope.

This behaviour is closely related to **closures**, which arise when a function retains access to variables from the environment in which it was created.

Closures are useful in some functional and API design patterns, although they are not required for most ordinary data science code.

1.5 Python's Argument-Passing Model

Descriptions such as *pass by value* and *pass by reference* can be misleading when applied directly to Python.

A more precise model begins with the fact that Python variables are **names bound to objects**.

Consider:

```
x = [1, 2, 3]
```

The name x refers to a list object. When x is supplied as an argument to a function, Python does not copy the list itself. Instead, the function parameter becomes another name bound to the same object.

For example:

```
def inspect_object(values):
    print(id(values))

numbers = [1, 2, 3]

print(id(numbers))
inspect_object(numbers)
```

```
2578625020352
2578625020352
```

Within the same Python process, both names refer to the same object.

This model is sometimes described as **call by sharing** or **passing object references by value**. The essential point is that the function receives access to the same object that was supplied by the caller.

Whether the caller observes a change depends on what the function subsequently does with that object.

1.5.1 Rebinding a Parameter

Consider a function that assigns a new value to one of its parameters:

```
def replace_value(x):
    x = 100
    print("Inside:", x)

value = 10

replace_value(value)
print("Outside:", value)
```

```
Inside: 100
Outside: 10
```

The assignment

```
x = 100
```

does not modify the object associated with value in the caller. It simply rebinds the local name x to another object.

The caller's name value remains bound to the original integer.

The same principle applies even when the original object is mutable:

```
def replace_list(values):
    values = [100, 200, 300]
    print("Inside:", values)

numbers = [1, 2, 3]

replace_list(numbers)
print("Outside:", numbers)
```

```
Inside: [100, 200, 300]
Outside: [1, 2, 3]
```

Again, the function merely rebinds its local parameter. The caller's variable remains unchanged.

1.5.2 Mutating an Object

The situation is different if the function **mutates** the object rather than rebinding the parameter:

```
def append_value(values):
    values.append(4)

numbers = [1, 2, 3]

append_value(numbers)
numbers
```

```
[1, 2, 3, 4]
```

The parameter values and the caller's name numbers refer to the same list object. Therefore, modifying that list through values is visible through numbers as well.

This distinction is fundamental:

> Rebinding a parameter changes what the local name refers to. Mutating an object changes the object itself.

The same distinction can be illustrated explicitly:

```
def rebind(values):
    values = [10, 20]

def mutate(values):
    values.append(10)

numbers = [1, 2, 3]

rebind(numbers)
print(numbers)

mutate(numbers)
print(numbers)
```

```
[1, 2, 3]
[1, 2, 3, 10]
```

rebind() does not affect the original list, whereas mutate() does.

1.6 Mutability and Function Behaviour

The argument-passing model becomes particularly important when distinguishing between **mutable** and **immutable** objects.

Common immutable Python objects include:

- integers;
- floating-point numbers;
- booleans;
- strings;
- tuples.

Common mutable objects include:

- lists;
- dictionaries;
- sets.

Because immutable objects cannot be modified in place, operations involving them necessarily produce other objects.

For example:

```
def increment(x):
    x += 1
    return x

value = 10

new_value = increment(value)

print(value)
print(new_value)
```

```
10
11
```

The integer object associated with value is not modified.

By contrast, a list can be modified in place:

```
def add_element(values):
    values.append(4)

numbers = [1, 2, 3]

add_element(numbers)

numbers
```

```
[1, 2, 3, 4]
```

A function that mutates one of its arguments therefore has an observable effect outside the function.

This is not inherently wrong. In-place modification can be intentional and useful. However, it should generally be clear from the function's purpose and documentation whether mutation is expected.

An alternative design is to create and return a new object:

```
def add_element(values):
    result = values.copy()
    result.append(4)
    return result

numbers = [1, 2, 3]
new_numbers = add_element(numbers)

print(numbers)
print(new_numbers)
```

```
[1, 2, 3]
[1, 2, 3, 4]
```

Now the original list remains unchanged.

The appropriate design depends on the context. Mutation can avoid unnecessary copies and may be appropriate for large data structures, while returning new values often makes data flow easier to reason about.

This distinction appears frequently in data science libraries. Some operations create new objects, while others modify existing objects in place. Understanding Python's object model makes those API behaviours easier to interpret.

1.7 Functions as Objects

Functions in Python are not merely pieces of syntax that can be called. They are objects in their own right.

Consider:

```
def square(x):
    return x ** 2
```

The name square is bound to a function object:

```
type(square)
```

```
function
```

Because functions are objects, they can be manipulated in many of the same ways as other Python values.

For example, a function can be assigned to another variable:

```
operation = square

operation(5)
```

```
25
```

The assignment does not call square. It binds the name operation to the same function object.

This distinction can be seen by comparing

```
operation = square
```

with

```
result = square(5)
```

The first expression refers to the function itself; the second calls the function and assigns its returned value.

1.7.1 Passing Functions as Arguments

Because functions are objects, they can be supplied as arguments to other functions.

For example:

```
def square(x):
    return x ** 2

def cube(x):
    return x ** 3

def apply_operation(operation, value):
    return operation(value)
```

We can then choose the operation dynamically:

```
apply_operation(square, 4)
```

```
16
```

```
apply_operation(cube, 4)
```

```
64
```

The parameter operation receives a function object. Inside apply_operation(), the expression

```
operation(value)
```

calls whichever function was provided.

A function that accepts another function as an argument, returns a function, or both is commonly called a **higher-order function**.

This idea appears throughout Python and is especially useful when an algorithm needs to receive some behaviour as input rather than having that behaviour hard-coded.

For example, Python's built-in sorted() function accepts a function through its key parameter:

```
words = ["pear", "watermelon", "fig", "apple"]

sorted(words, key=len)
```

```
['fig', 'pear', 'apple', 'watermelon']
```

Here, len is passed as a function object. sorted() calls it internally for each element and uses the resulting values to determine the ordering.

Notice that we write

```
key=len
```

rather than

```
key=len()
```

The first passes the function itself. The second would attempt to call len immediately without supplying the required argument.

1.7.2 Returning Functions

A function can also create and return another function.

For example:

```
def make_multiplier(factor):

    def multiply(x):
        return x * factor

    return multiply
```

Calling make_multiplier() returns a function:

```
double = make_multiplier(2)
triple = make_multiplier(3)
```

Those returned functions can subsequently be called:

```
double(10)
```

```
20
```

```
triple(10)
```

```
30
```

The interesting feature is that multiply() continues to have access to factor even after make_multiplier() has finished executing.

The function therefore retains part of the environment in which it was created. This is a **closure**.

In the example above:

- double retains factor = 2;
- triple retains factor = 3.

Closures provide a way to create functions whose behaviour is configured by previously supplied values.

1.7.3 Storing Functions in Data Structures

Functions can also be stored in collections such as lists or dictionaries.

For example:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

operations = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply
}
```

An operation can then be selected dynamically:

```
operation = operations["multiply"]

operation(4, 5)
```

```
20
```

Or, more compactly:

```
operations["add"](10, 5)
```

```
15
```

This pattern illustrates an important consequence of treating functions as ordinary objects: program behaviour can itself be represented as data.

Rather than constructing a long sequence of conditional statements, a program can sometimes map names directly to functions:

```
def calculate(operation, a, b):
    return operations[operation](a, b)

calculate("subtract", 10, 4)
```

```
6
```

This technique appears in dispatch systems, data-processing pipelines, application frameworks, and configurable machine learning workflows.

The ability to assign functions to names, pass them as arguments, return them from other functions, and store them in collections is summarized by saying that Python functions are **first-class objects**.

This property provides the conceptual foundation for several features considered next, including lambda expressions and higher-order functional operations.

1.8 Lambda Functions

Python provides **lambda expressions** as a compact way to create small anonymous functions.

The general form is

```
lambda parameters: expression
```

For example, the function

```
def square(x):
    return x ** 2
```

can also be written as

```
square = lambda x: x ** 2
```

Both objects can be called in the same way:

```
square(5)
```

```
25
```

A lambda expression evaluates to a function object. The main difference is syntactic: a lambda contains a single expression rather than a conventional function body composed of one or more statements.

For example:

```
add = lambda a, b: a + b

add(3, 4)
```

```
7
```

Lambda expressions are often described as **anonymous functions** because they can be created without first assigning them a name.

For example:

```
(lambda x: x ** 2)(5)
```

```
25
```

Here, the function is created and immediately called.

In practice, lambda expressions are most useful when a short function is required temporarily as an argument to another function.

For example, consider sorting a collection of tuples according to their second element:

```
records = [
    ("Alice", 82),
    ("Bob", 91),
    ("Carol", 76)
]

sorted(records, key=lambda record: record[1])
```

```
[('Carol', 76), ('Alice', 82), ('Bob', 91)]
```

The expression

```
lambda record: record[1]
```

defines the transformation that sorted() should use to determine the ordering.

The equivalent named function would be

```
def get_score(record):
    return record[1]

sorted(records, key=get_score)
```

```
[('Carol', 76), ('Alice', 82), ('Bob', 91)]
```

Both approaches are valid.

A lambda is usually appropriate when the operation is:

- short;
- used in a limited context;
- immediately understandable;
- unlikely to require documentation or independent testing.

If the logic becomes more complex, a named function is generally preferable.

For example, this is technically valid:

```
classification = lambda x: "positive" if x > 0 else "zero" if x == 0 else "negative"
```

but a regular function is easier to read and maintain:

```
def classify_number(x):
    if x > 0:
        return "positive"

    if x == 0:
        return "zero"

    return "negative"
```

Lambda expressions should therefore be treated as a convenience for compact function definitions rather than as a replacement for ordinary functions.

1.9 Higher-Order Functions

A **higher-order function** is a function that accepts another function as an argument, returns a function, or both.

Python's support for first-class functions makes higher-order functions a natural part of the language.

Several built-in functions use this pattern.

1.9.1 map()

The built-in map() function applies a function to each element of an iterable.

For example:

```
numbers = [1, 2, 3, 4, 5]

squared = map(lambda x: x ** 2, numbers)

squared
```

<map at 0x25862051450>

map() does not immediately return a list. It returns a map object that produces values lazily.

The results can be materialized as a list:

```
list(squared)
```

[1, 4, 9, 16, 25]

The same operation can be written using a named function:

```
def square(x):
    return x ** 2

list(map(square, numbers))
```

[1, 4, 9, 16, 25]

or, more idiomatically in many Python programs, using a list comprehension:

```
[x ** 2 for x in numbers]
```

```
[1, 4, 9, 16, 25]
```

All three forms represent essentially the same element-wise transformation.

List comprehensions are often preferred when the transformation is simple because they express both the iteration and transformation directly.

However, map() remains useful when a function object already exists or when working within a functional programming style.

For example:

```
values = ["1", "2", "3", "4"]

list(map(int, values))
```

```
[1, 2, 3, 4]
```

Here, the existing int function is passed directly to map().

1.9.2 filter()

The built-in filter() function retains elements for which a predicate returns a truthy value.

A **predicate** is a function representing a condition, usually returning True or False.

For example:

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = filter(lambda x: x % 2 == 0, numbers)

list(even_numbers)
```

```
[2, 4, 6]
```

Again, the equivalent operation can be expressed with a list comprehension:

```
[x for x in numbers if x % 2 == 0]
```

```
[2, 4, 6]
```

For relatively simple conditions, the comprehension is often easier to read.

A named predicate may be preferable when the condition has independent meaning:

```
def is_even(x):
    return x % 2 == 0

list(filter(is_even, numbers))
```

```
[2, 4, 6]
```

This version expresses the intent explicitly through the name is_even.

1.9.3 sorted() and the key Function

As seen previously, sorted() is another important higher-order function because it can receive a function through the key parameter.

Consider:

```
words = ["pear", "watermelon", "fig", "apple"]

sorted(words, key=len)
```

```
['fig', 'pear', 'apple', 'watermelon']
```

The function len is applied to each element, and the resulting values determine the ordering.

A lambda can define a custom sorting criterion:

```
records = [
    {"name": "Alice", "score": 82},
    {"name": "Bob", "score": 91},
    {"name": "Carol", "score": 76}
]

sorted(records, key=lambda record: record["score"])
```

```
[{'name': 'Carol', 'score': 76},
 {'name': 'Alice', 'score': 82},
 {'name': 'Bob', 'score': 91}]
```

Descending order can be requested using reverse=True:

```
sorted(
    records,
    key=lambda record: record["score"],
    reverse=True
)
```

```
[{'name': 'Bob', 'score': 91},
 {'name': 'Alice', 'score': 82},
 {'name': 'Carol', 'score': 76}]
```

The key pattern is common in Python APIs because it allows an algorithm to separate **how values are processed** from **which operation defines the relevant criterion**.

1.9.4 min() and max() with key

The same idea appears in min() and max().

For example:

```
max(records, key=lambda record: record["score"])
```

```
{'name': 'Bob', 'score': 91}
```

This returns the complete record associated with the largest score rather than only the score itself.

Similarly:

```
min(words, key=len)
```

```
'fig'
```

returns the shortest word.

These examples illustrate a broader design principle:

A function can make an algorithm configurable by receiving behaviour through another function.

This is often more flexible than hard-coding every possible variation into the algorithm itself.

1.10 Comprehensions and Functional Tools

Operations performed with map() and filter() can frequently be expressed using comprehensions.

For example:

```
numbers = [1, 2, 3, 4, 5, 6]
```

Transformation with map():

```
list(map(lambda x: x ** 2, numbers))
```

```
[1, 4, 9, 16, 25, 36]
```

Equivalent list comprehension:

```
[x ** 2 for x in numbers]
```

```
[1, 4, 9, 16, 25, 36]
```

Filtering with filter():

```
list(filter(lambda x: x % 2 == 0, numbers))
```

```
[2, 4, 6]
```

Equivalent list comprehension:

```
[x for x in numbers if x % 2 == 0]
```

```
[2, 4, 6]
```

Transformation and filtering can also be combined:

```
[x ** 2 for x in numbers if x % 2 == 0]
```

[4, 16, 36]

The equivalent combination using map() and filter() is

```
list(
    map(
        lambda x: x ** 2,
        filter(lambda x: x % 2 == 0, numbers)
    )
)
```

[4, 16, 36]

Although both implementations are valid, the comprehension is generally easier to read in this case.

The choice between comprehensions and higher-order functional tools should therefore be guided primarily by clarity rather than by adherence to a particular programming style.

1.11 Docstrings

Functions should communicate not only their implementation but also their intended interface and behaviour.

Python provides **docstrings** for documenting functions directly in the source code.

A docstring is a string literal placed immediately after the function definition:

```
def mean(values):
    """Return the arithmetic mean of a non-empty sequence."""
    return sum(values) / len(values)
```

The docstring becomes part of the function object and can be inspected through __doc__:

```
mean.__doc__
```

'Return the arithmetic mean of a non-empty sequence.'

or using Python's built-in help system:

```
help(mean)
```

For simple functions, a concise one-line docstring may be sufficient.

More substantial functions often benefit from documentation describing:

- the purpose of the function;
- its parameters;
- its return value;
- important assumptions;
- possible exceptions;
- relevant side effects.

For example:

```
def normalize(values):
    """
    Normalize a sequence of numeric values by their total.

    Parameters
    ----------
    values : sequence of float
        Numeric values to normalize.

    Returns
    -------
    list of float
        Values divided by their total.

    Raises
    ------
    ValueError
        If the total is zero.
    """
    total = sum(values)

    if total == 0:
        raise ValueError("Cannot normalize values with zero total.")

    return [value / total for value in values]
```

This style resembles the documentation conventions widely used in scientific Python libraries.

The important principle is that documentation should describe the **contract** of the function rather than merely restating its implementation.

For example, the comment

```
# Divide every value by the total
```

adds relatively little information to

```
return [value / total for value in values]
```

A more useful explanation would describe why normalization is being performed, what assumptions are required, or how the function behaves in edge cases.

1.12 Type Hints

Python supports **type annotations** that allow the expected types of parameters and return values to be expressed explicitly.

For example:

```
def square(x: float) -> float:
    return x ** 2
```

The annotation

```
x: float
```

indicates that x is expected to be a floating-point value, while

```
-> float
```

indicates that the function is expected to return a floating-point value.

Annotations can also describe collections:

```
def mean(values: list[float]) -> float:
    return sum(values) / len(values)
```

Multiple parameters can be annotated independently:

```
def power(base: float, exponent: int) -> float:
    return base ** exponent
```

Default values and annotations can be combined:

```
def power(base: float, exponent: int = 2) -> float:
    return base ** exponent
```

An important point is that Python does not normally enforce these annotations at runtime.

For example:

```
def add(a: int, b: int) -> int:
    return a + b
```

Python will still allow:

```
add("Hello, ", "world!")
```

```
'Hello, world!'
```

because the annotations themselves do not automatically perform runtime type checking.

Type hints instead provide information for:

- developers reading the code;
- IDEs and editors;
- static type checkers;
- documentation tools;
- automated analysis tools.

They therefore improve the explicitness of a function's interface without changing Python's fundamentally dynamic type system.

1.12.1 Optional Values

A function may intentionally accept None as well as another type.

Using modern Python syntax, this can be expressed as:

```
def append_value(
    value: int,
    values: list[int] | None = None
) -> list[int]:

    if values is None:
        values = []

    values.append(value)

    return values
```

The expression

```
list[int] | None
```

indicates that values may refer either to a list of integers or to None.

This annotation makes explicit the sentinel pattern used earlier to avoid mutable default arguments.

1.12.2 Type Hints as Interface Documentation

Type hints are most valuable when they clarify how a function is intended to be used.

Compare:

```
def calculate(x, y, option):
    ...
```

with

```
def calculate(
    x: float,
    y: float,
    option: str
) -> float:
    ...
```

The second function communicates considerably more information before its implementation is even inspected.

However, type hints should not be treated as a substitute for good naming or documentation. A function can be precisely annotated and still have an unclear purpose.

The strongest interfaces combine:

- meaningful names;
- appropriate type hints;
- concise documentation;
- predictable behaviour.

1.13 Pure Functions and Side Effects

A useful distinction when designing functions is the difference between **pure functions** and functions with **side effects**.

A pure function has two principal characteristics:

1. its result depends only on its input values;
2. evaluating it does not modify observable state outside the function.

For example:

```
def square(x):
    return x ** 2
```

Given the same value of x, this function always produces the same result, and it does not modify any external state.

Another example is:

```
def standardize(x, mean, std):
    return (x - mean) / std
```

All information required for the computation is supplied explicitly through the parameters.

Pure functions are often easier to:

- reason about;
- test;
- reuse;
- debug;

- compose into larger computations.

By contrast, consider:

```
counter = 0

def increment_counter():
    global counter
    counter += 1
```

Calling increment_counter() changes state outside the function. This is a **side effect**.

Other common side effects include:

- modifying mutable arguments;
- writing to a file;
- modifying a database;
- printing output;
- sending a network request;
- changing global state.

Side effects are not inherently undesirable. Many useful programs ultimately exist precisely to produce them.

For example:

```
print("Training complete")
```

has value because it produces an observable effect.

Similarly, saving a trained machine learning model to disk necessarily changes external state.

The design question is therefore not whether all side effects should be eliminated, but whether they should be made explicit and controlled.

For example, compare:

```
results = []

def compute_square(x):
    results.append(x ** 2)
```

with

```
def compute_square(x):
    return x ** 2

results = []
results.append(compute_square(5))
```

The second version separates the computation from the mutation of results. The flow of information is therefore easier to see.

This separation is particularly useful in data-processing and machine learning workflows, where computational transformations can often be kept independent from operations such as reading data, writing results, logging, or updating external systems.

1.14 Recursion

A function is **recursive** when it calls itself directly or indirectly.

Recursive functions are useful when a problem can naturally be described in terms of smaller instances of the same problem.

A classical example is the factorial function.

For a non-negative integer (n),

$$n! = \begin{cases} 1, & n = 0, \\ n(n-1)!, & n > 0. \end{cases} \tag{2}$$

This definition is itself recursive because $(n!)$ is defined in terms of $((n\text{-}1)!)$.

The corresponding Python implementation is:

```
def factorial(n):
    if n == 0:
        return 1

    return n * factorial(n - 1)
```

For example:

```
factorial(5)
```

```
120
```

The evaluation conceptually expands as

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1!.$$

Eventually the recursion reaches the condition that terminates the process.

This terminating condition is called the **base case**.

Without a valid base case, recursion continues until Python reaches its recursion limit:

```
def infinite_recursion():
    return infinite_recursion()
```

A recursive function therefore normally contains at least:

- a **base case**, which terminates the recursion;
- a **recursive case**, which reduces the problem toward the base case.

1.14.1 Recursion Versus Iteration

Many recursive problems can also be solved iteratively.

For example, factorial can be implemented as:

```
def factorial_iterative(n):
    result = 1

    for value in range(2, n + 1):
        result *= value

    return result
```

Both implementations produce the same mathematical result:

```
factorial(5) == factorial_iterative(5)
```

```
True
```

In Python, iteration is often preferable for simple repetitive computations because each recursive call introduces an additional function call and consumes stack space.

Recursion becomes more attractive when the structure of the problem is itself recursive, as can occur with:

- trees;
- nested data structures;
- divide-and-conquer algorithms;
- recursive mathematical definitions.

The goal is not to prefer recursion or iteration universally, but to choose the representation that makes the underlying structure of the problem clearest.

1.15 Designing Functions

Writing a syntactically valid function is straightforward. Designing a function that remains clear and useful as a program grows requires additional judgement.

A well-designed function generally represents **one coherent responsibility**.

For example, a data science workflow might initially be written as one large function:

```
def run_analysis(...):
    # load data
    # clean data
    # create features
    # fit model
    # calculate metrics
    # save results
    ...
```

Although this may work, several distinct responsibilities have been combined.

A clearer design may decompose the workflow into functions such as:

```
def load_data(...):
    ...

def clean_data(...):
    ...

def create_features(...):
    ...

def fit_model(...):
    ...

```

```
def evaluate_model(...):
    ...
```

A higher-level function can then coordinate them:

```
def run_analysis(...):
    data = load_data(...)
    clean_data = clean_data(data)
    features = create_features(clean_data)
    model = fit_model(features)
    metrics = evaluate_model(model, features)

    return model, metrics
```

The precise decomposition depends on the problem. Functions should not be split merely to make them shorter.

A useful function creates a meaningful abstraction.

1.15.1 Prefer Explicit Inputs and Outputs

Whenever practical, the information required by a computation should be visible in the function interface.

Compare:

```
threshold = 0.5

def classify(probability):
    return probability >= threshold
```

with:

```
def classify(probability, threshold=0.5):
    return probability >= threshold
```

The second design makes the dependency explicit and allows the behaviour to be configured by the caller.

This generally improves testability and reuse.

1.15.2 Use Descriptive Names

A function name should communicate the operation being performed.

For example:

```
def f(x):
    ...
```

provides little information outside a narrow mathematical context.

A name such as

```
def calculate_accuracy(predictions, targets):
    ...
```

communicates substantially more intent.

Names commonly use verbs or verb phrases because functions usually represent actions or computations:

```
load_data()
calculate_mean()
fit_model()
predict_probabilities()
validate_input()
```

The level of specificity should correspond to the abstraction represented by the function.

1.15.3 Avoid Unnecessary Boolean Arguments

Boolean parameters can sometimes make calls difficult to interpret.

For example:

```
process_data(data, True, False)
```

does not reveal what the boolean values mean.

Keyword arguments improve the situation:

```
process_data(
    data,
    normalize=True,
    remove_outliers=False
)
```

However, if a boolean substantially changes the responsibility of the function, separate functions may sometimes provide a clearer interface.

For example:

```
save_model(model, compressed=True)
```

may be entirely reasonable, while a function whose behaviour changes completely according to several boolean switches may be trying to represent too many operations at once.

1.15.4 Avoid Hidden Mutation

If a function modifies one of its arguments, that behaviour should be intentional and predictable.

For example:

```
def normalize(values):
    ...
```

does not make it immediately obvious whether values will be modified.

If the function returns a new collection, its behaviour is easier to infer:

```
normalized_values = normalize(values)
```

When mutation is required for performance or API reasons, documentation and naming should make that behaviour clear.

1.15.5 Separate Computation from I/O When Practical

Functions that perform calculations are often easier to test when separated from input/output operations.

Instead of:

```
def calculate_and_print_mean(values):
    mean = sum(values) / len(values)
    print(mean)
```

prefer, where appropriate:

```
def calculate_mean(values):
    return sum(values) / len(values)

mean = calculate_mean([1, 2, 3, 4])
print(mean)
```

```
2.5
```

The computational function can now be tested independently from how its result is displayed.

The same principle extends to operations such as:

- loading files;
- saving models;
- querying databases;
- logging;
- generating reports.

Separating computational logic from external interactions often produces code that is easier to reuse and verify.

1.15.6 Helper Functions

A complex operation can often be decomposed using smaller **helper functions**.

For example:

```python
def is_valid_probability(value):
    return 0 <= value <= 1


def validate_probabilities(probabilities):
    return all(
        is_valid_probability(value)
        for value in probabilities
    )
```

is_valid_probability() exists primarily to support the higher-level validation operation.

A helper function is not a fundamentally different kind of function. It is simply a function whose role is to encapsulate a smaller piece of behaviour within a larger design.

The important question is whether the helper creates a meaningful abstraction or eliminates unnecessary duplication.

1.16 Key Takeaways

Functions are one of the central abstraction mechanisms in Python.

A function defines a reusable operation with an interface consisting primarily of its parameters and return value. Parameters describe the inputs accepted by the function, while arguments provide concrete values during a particular call.

Python supports several mechanisms for constructing flexible function interfaces, including positional arguments, keyword arguments, default values, *args, and **kwargs.

Names defined inside functions follow Python's scope rules. Name resolution can be summarized by the **LEGB** hierarchy: Local, Enclosing, Global, and Built-in.

Python's argument-passing behaviour is best understood in terms of objects and references. Function parameters become local names bound to the objects supplied by the caller. Rebinding a parameter does not modify the caller's binding, whereas mutating a shared mutable object can produce externally visible changes.

Functions are **first-class objects**. They can be assigned to variables, passed as arguments, returned by other functions, and stored in data structures. This provides the basis for higher-order functions, closures, and lambda expressions.

Lambda expressions provide a compact representation of simple functions, particularly when a short operation needs to be supplied to another function. For more complex logic, ordinary named functions are generally clearer.

Built-in operations such as map(), filter(), sorted(), min(), and max() illustrate the use of functions as inputs to other functions. In many transformation and filtering tasks, comprehensions provide an alternative that is often particularly readable in Python.

Docstrings describe a function's behaviour and contract, while type hints make the intended types of its inputs and outputs more explicit. Type annotations improve interfaces and tooling but do not normally enforce types at runtime.

Pure functions make dependencies and data flow explicit by computing results without modifying external state. Side effects are often necessary, but separating them from computational logic can improve reasoning, testing, and reuse.

Recursion provides a natural representation for problems that can be decomposed into smaller instances of themselves, although iteration is often preferable for straightforward repetitive computations in Python.

Ultimately, good function design is less about syntax than about **abstraction and interfaces**. A useful function isolates a coherent operation, makes its dependencies explicit, produces predictable behaviour, and allows a larger program to be understood in terms of meaningful components rather than implementation details.

2 Problems

The following problems review the main concepts introduced in this chapter. They progress from basic function definition and argument handling to scope, mutability, higher-order functions, and function design.

2.1 Problem 1 — Basic Function Definition

Write a function rectangle_area() that receives the width and height of a rectangle and returns its area.

Use the function to calculate the area of a rectangle with width 7.5 and height 4.

Then explain the roles of the parameters, arguments, and return value in the function call.

2.2 Problem 2 — Positional and Keyword Arguments

Consider the following function:

```
1 def calculate_total(price, quantity, discount=0):
2     return price * quantity * (1 - discount)
```

Evaluate the following calls and determine whether each one is valid:

```
calculate_total(10, 5)
calculate_total(10, 5, 0.2)
calculate_total(price=10, quantity=5, discount=0.2)
calculate_total(quantity=5, price=10)
calculate_total(10, quantity=5)
```

For every valid call, determine the value associated with each parameter and the value returned by the function.

2.3 Problem 3 — Default Arguments

Write a function calculate_power() that receives a numeric value and an optional exponent.

The exponent should default to 2.

The following calls should therefore be valid:

```
calculate_power(5)
calculate_power(5, 3)
calculate_power(5, exponent=4)
```

Explain how Python determines the value of exponent in each case.

2.4 Problem 4 — Mutable Default Arguments

Consider:

```
def add_observation(value, observations=[]):
    observations.append(value)
    return observations
```

Without executing the code, predict the results of:

```
add_observation(10)
add_observation(20)
add_observation(30)
```

Explain why this behaviour occurs.

Then rewrite the function so that a new list is created whenever the caller does not explicitly provide one.

2.5 Problem 5 — Variable-Length Positional Arguments

Write a function calculate_mean() that accepts an arbitrary number of positional numeric arguments using *args and returns their arithmetic mean.

For example:

```
calculate_mean(10, 20, 30)
calculate_mean(1, 2, 3, 4, 5)
```

What type of object does Python use internally to store the additional positional arguments?

How should the function behave if no arguments are supplied?

2.6 Problem 6 — Variable-Length Keyword Arguments

Write a function model_configuration() that accepts arbitrary keyword arguments using **kwargs and returns them as a dictionary.

For example:

```
model_configuration(
    model="random_forest",
    n_estimators=200,
    max_depth=10
)
```

Explain the relationship between the keyword names in the function call and the resulting dictionary.

2.7 Problem 7 — Return Values

Write a function descriptive_range() that receives a sequence of numeric values and returns both its minimum and maximum values.

For example:

```
minimum, maximum = descriptive_range([4, 8, 1, 9, 3])
```

Explain why it is common to say that the function returns two values, even though Python technically returns a single object.

What is the type of that object?

2.8 Problem 8 — Implicit Return Value

Consider:

```
def display_result(value):
    print(f"Result: {value}")
```

What value is returned by:

```
result = display_result(10)
```

Explain the difference between the value printed by the function and the value returned by the function.

Why can confusing these two concepts lead to errors when composing several functions?

2.9 Problem 9 — Scope and Name Resolution

Without executing the code, determine what will be printed:

```
x = 10

def outer():
    x = 20

    def inner():
        x = 30
        print(x)

    inner()
    print(x)

outer()
print(x)
```

Explain the result using Python's LEGB rule.

2.10 Problem 10 — Enclosing Scope

Consider:

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        print(x)

    inner()

outer()
```

Which x does inner() use?

Explain how Python searches for the name and identify each relevant scope.

2.11 Problem 11 — Rebinding Versus Mutation

Predict the output of the following program without executing it:

```
def modify(x, values):
    x = x + 1
    values.append(x)

number = 10
numbers = [1, 2, 3]

modify(number, numbers)

print(number)
print(numbers)
```

Explain why number and numbers behave differently.

Your explanation should distinguish between:

- rebinding a local parameter;

- mutating an existing object;
- mutable and immutable objects.

2.12 Problem 12 — Object Identity

Consider:

```
def inspect(values):
    print(id(values))

numbers = [1, 2, 3]

print(id(numbers))
inspect(numbers)
```

What relationship do you expect between the two values printed by id()?

What does this tell you about the object associated with the parameter values?

Use this result to explain why describing Python simply as either “pass by value” or “pass by reference” can be misleading.

2.13 Problem 13 — Avoiding Unintended Mutation

Consider:

```
def remove_negative_values(values):
    for value in values.copy():
        if value < 0:
            values.remove(value)
```

The function modifies the list supplied by the caller.

Rewrite the function so that it returns a new list containing only non-negative values and leaves the original list unchanged.

Then compare the two designs.

Under what circumstances might mutation be intentional?

2.14 Problem 14 — Functions as First-Class Objects

Consider:

```
def square(x):
    return x ** 2

operation = square
```

Without executing the code, determine the difference between:

```
operation
operation(5)
```

Then explain the difference between:

```
operation = square
```

and

```
operation = square(5)
```

What property of Python functions makes the first assignment possible?

2.15 Problem 15 — Passing Behaviour as an Argument

Write the following two functions:

```
def square(x):
    ...

def cube(x):
    ...
```

Then write a function

```
def apply_operation(operation, values):
    ...
```

that receives another function and a sequence of values and returns a list containing the result of applying the supplied function to every value.

For example:

```
apply_operation(square, [1, 2, 3, 4])
apply_operation(cube, [1, 2, 3, 4])
```

Explain why apply_operation() is a higher-order function.

2.16 Problem 16 — Lambda Expressions

Rewrite the following function as a lambda expression:

```
def squared_difference(x, y):
    return (x - y) ** 2
```

Then use a lambda expression to sort the following records by score:

```
records = [
    ("Alice", 82),
    ("Bob", 91),
    ("Carol", 76),
    ("David", 88)
]
```

Sort them first in ascending order and then in descending order.

2.17 Problem 17 — map(), filter(), and Comprehensions

Given:

```
values = [-3, -2, -1, 0, 1, 2, 3, 4]
```

Produce a collection containing the squares of only the positive values.

Solve the problem in two ways:

1. using filter() and map();
2. using a list comprehension.

Compare the two implementations in terms of readability.

2.18 Problem 18 — Functions as Sorting Criteria

Suppose that a set of model results is represented as:

```
models = [
    {"name": "model_a", "rmse": 3.4},
    {"name": "model_b", "rmse": 2.8},
    {"name": "model_c", "rmse": 3.1}
]
```

Use min() with an appropriate key function to select the model with the lowest RMSE.

Then use sorted() to order all models from lowest to highest RMSE.

Explain why the complete dictionary is returned rather than only the RMSE value.

2.19 Problem 19 — Docstrings and Type Hints

Write a function with the following interface:

```
def accuracy(
    predictions: list[int],
    targets: list[int]
) -> float:
    ...
```

The function should calculate the proportion of predictions that equal their corresponding targets.

Add an appropriate docstring describing:

- the purpose of the function;
- its parameters;
- its return value.

What additional validation might be appropriate before performing the calculation?

2.20 Problem 20 — Pure Functions and Side Effects

Consider the following two implementations:

```
results = []

def square_a(x):
    results.append(x ** 2)
```

and

```
def square_b(x):
    return x ** 2
```

Compare their behaviour.

Which function is pure?

Which function produces a side effect?

Explain why the second implementation is generally easier to test and compose with other functions.

2.21 Problem 21 — Recursion

Write a recursive function sum_to_n() that calculates

$$1 + 2 + \cdots + n$$

for a positive integer (n).

Identify explicitly:

- the base case;
- the recursive case;
- how each recursive call moves the computation toward termination.

Then write an iterative version of the same function and compare the two implementations.

2.22 Problem 22 — Designing a Data Transformation Function

Suppose a dataset contains a numeric feature represented as a Python list:

```
values = [12.0, 15.0, 18.0, 21.0, 24.0]
```

Write a function

```
def standardize(values: list[float]) -> list[float]:
    ...
```

that calculates

$$z_i = \frac{x_i - \bar{x}}{s}, \tag{3}$$

where $\bar{x}$ is the arithmetic mean and s is the sample standard deviation.

The function should:

- leave the original list unchanged;
- return a new list;
- use explicit inputs and outputs;
- include appropriate type hints;
- include a concise docstring.

Consider how the function should behave if the standard deviation is zero.

2.23 Problem 23 — Refactoring a Machine Learning Workflow

Consider the following function:

```
def run_model(file_path):
    # load the dataset
    # remove missing observations
    # create model features
    # train the model
    # generate predictions
    # calculate evaluation metrics
    # print the metrics
    # save the model
    ...
```

Critique this design.

Identify the different responsibilities contained in the function and propose a decomposition into smaller functions.

Your answer should consider:

- abstraction;
- explicit inputs and outputs;
- separation of computation from I/O;
- testability;
- side effects.

There is no single correct decomposition. The objective is to justify the boundaries chosen between functions.

2.24 Problem 24 — Reasoning About an Unfamiliar Function

Consider:

```
def process(values, transform=lambda x: x, condition=lambda x: True):
    return [
        transform(value)
        for value in values
        if condition(value)
    ]
```

Without executing the code, determine the results of:

```
process([1, 2, 3, 4])

process(
    [1, 2, 3, 4],
    transform=lambda x: x ** 2
)

process(
    [1, 2, 3, 4],
    transform=lambda x: x ** 2,
    condition=lambda x: x % 2 == 0
)
```

Then explain the function in conceptual terms.

In particular, identify:

- the role of each parameter;
- the purpose of the default lambda expressions;
- which arguments represent data and which represent behaviour;
- why process() can be considered a higher-order function.

Finally, describe one situation in a data-processing or machine learning workflow where this general design pattern could be useful.